

Day 11 — Robustezza e valutazione

Il Day 10 ci ha lasciato un assistente funzionante con nove flow e la coesistenza NLU/LLM. Oggi renderai più sicuro il flow di trasferimento e trasformerai le conversazioni importanti in test end-to-end ripetibili.

Farai quanto segue:

1. osserverai i pattern di riparazione della conversazione di default;
2. impedirai le interruzioni durante l'autenticazione;
3. personalizzerai il chitchat;
4. validerai i dati del trasferimento su tre livelli differenti;
5. creerai una suite di regressione end-to-end e ne ispezionerai la coverage.

Lavora all'interno della cartella `starting-point/` di questa lezione. Il progetto completato è disponibile in `end-result/` per un confronto.

1. Prepara il progetto

Ti servono:

- Python 3.11 e `uv` ;
- `RASA_LICENSE` esportata nella tua shell;
- `OPENAI_API_KEY` esportata nella tua shell.

Il Makefile fornisce queste impostazioni dimostrative all'assistente e alla mock bank:

```
CORE_BANKING_URL=http://localhost:8000
CORE_BANKING_TOKEN=test_token
```

Da `starting-point/` , crea gli ambienti una volta sola:

```
make setup
```

Usa poi due terminali, entrambi aperti in `starting-point/` :

```
# Terminal 1: mock core-banking API
make run-fastapi-server
```

```
# Terminal 2: train the assistant and open Inspector
make run-rasa
```

Il primo training e i successivi test end-to-end chiamano il modello OpenAI configurato. Un'esecuzione end-to-end completa effettua diverse decine di chiamate LLM. Con la modalità summary di default, ogni

risposta riformulata può comportare due chiamate LLM: una per riassumere la storia della conversazione e una per riformulare la risposta.

Ogni nuova conversazione inizia non autenticata. Usa `123456` quando l'assistente chiede il codice di sicurezza. Tieni visibile il pannello dello stack dell'Inspector mentre testi le interruzioni.

2. Osserva i pattern di riparazione di default

Prima di modificare il progetto, prova le conversazioni seguenti. L'esatto routing generato dall'LLM o la formulazione esatta possono variare; ispeziona lo stack e gli eventi anziché aspettarti un testo identico.

Interrompi e riprendi un trasferimento

Avvia un trasferimento, poi chiedi il tuo saldo:

```
user: I want to send some money to a friend
bot: For your security, please enter your one-time code.
user: 123456
bot: Who is the recipient of the transfer?
user: Wait, how much is in my checking account?
bot: Which account would you like to check?
user: checking
bot: Your checking account balance is €1250.0.
bot: Would you like to continue with transfer money?
user: yes
bot: Who is the recipient of the transfer?
```

`check_balance` viene spinto sopra il trasferimento messo in pausa. Una volta completato, `pattern_continue_interrupted` chiede se riprendere il trasferimento.

Innesca la clarification

Durante un trasferimento, invia una richiesta ambigua riguardante la carta:

```
user: Something's wrong with my card
bot: I can help, but I need more information. Which of these would you like to do:
block a card or replace a card?
```

Quando diversi flow sono plausibili, `pattern_clarification` presenta i candidati. La lista dei candidati può variare, perché è l'LLM a sceglierla.

Correggi un valore raccolto

Raggiungi il recap del trasferimento con un importo di `200`, poi scrivi:

```
user: Actually, make it 150 euros
bot: Ok, I am updating transfer_amount to 150.0.
bot: Please confirm: transfer €150.0 to Marco Bianchi, IBAN IT60...3456. Shall I proceed?
```

`pattern_correction` aggiorna lo slot. Il recap viene richiesto di nuovo perché il suo step di collect usa `ask_before_filling: true`.

Annulla il flow attivo

```
user: Forget it, cancel the whole thing
bot: Okay, stopping transfer money.
bot: Is there anything else I can help you with?
```

`pattern_cancel_flow` rimuove il trasferimento dallo stack. `pattern_completed` chiude poi l'interazione.

Ferma il backend prima della conferma

Raggiungi il recap, ferma il Terminal 1 con Ctrl-C e conferma il trasferimento:

```
bot: I couldn't reach the transfer system, so no money has moved. Please try again later.
```

Questa risposta non proviene da un repair pattern. `action_execute_transfer` gestisce l'errore di connessione e il flow segue il suo branch di fallimento. Questo è più sicuro che affidarsi a `pattern_internal_error`, che può soltanto segnalare un generico guasto software.

Riavvia la mock bank prima di proseguire:

```
make run-fastapi-server
```

3. Blocca le interruzioni durante l'autenticazione

Una richiesta di codice di sicurezza non dovrebbe consentire digressioni. Modifica:

```
lumera-assistant/data/flows/shared/authenticate_user.yml
```

Sostituisci lo step di collect di `security_code` con:

```
- collect: security_code
  id: ask_code
  description: the one-time security code the customer provides
  ask_before_filling: true
  # While the code is being collected, ignore every command except
  # filling this slot – no digressions during a security challenge.
  force_slot_filling: true
```

`force_slot_filling: true` ignora ogni comando eccetto quello che riempie questo slot, finché lo step di collect è attivo.

Riavvia l'assistente con un nuovo training:

```
make run-rasa
```

Chiedi il saldo mentre l'assistente attende il codice di sicurezza:

```
bot: For your security, please enter your one-time code.
user: Wait, how much is in my checking account?
bot: I'm sorry I am unable to understand you, could you please rephrase?
bot: For your security, please enter your one-time code.
```

Il flow del saldo non parte più. Usa questa impostazione solo dove le interruzioni sono pericolose o prive di senso.

4. Personalizza il chitchat

Il `pattern_chitchat` di default rifiuta le richieste fuori tema. Sovrascrivilo con la risposta di free-chitchat fornita di serie.

Crea:

```
lumera-assistant/data/flows/patterns/pattern_chitchat.yml
```

```
flows:
  # Override of the shipped chitchat pattern: a flow with the reserved name
  # replaces the default (which answers utter_cannot_handle). The free-form
  # response only becomes generative once the rephraser is wired in
  # endpoints.yml – without it, the static fallback text is served.
pattern_chitchat:
  description: handle interactions with the user that are not task-oriented
  name: pattern chitchat
  steps:
    - action: utter_free_chitchat_response
```

Un flow con questo ID riservato sostituisce il pattern di default. Riallena e riavvia:

```
make run-rasa
```

Prova una richiesta fuori tema durante un trasferimento:

```
user: By the way, do you like pizza?  
bot: Sorry, I'm not able to answer that right now.
```

Questo è ancora testo di fallback statico. `utter_free_chitchat_response` è marcato per la riformulazione, ma il progetto non ha ancora un rephraser.

Modifica `lumera-assistant/endpoints.yml`. Aggiungi questo blocco dopo `action_endpoint`:

```
# Contextual Response Rephraser: rephrases responses that carry  
# `metadata: rephrase: True` (our own responses don't – only the shipped  
# pattern responses and utter_free_chitchat_response do). Endpoints are  
# runtime configuration: changing this file needs a restart, not a retrain.  
nlg:  
  type: rephrase  
  llm:  
    model_group: openai_llm
```

`endpoints.yml` è configurazione di runtime, quindi riavvia l'assistente senza riallenarlo. Da `lumera-assistant/` nel Terminal 2, esegui:

```
.venv/bin/rasa inspect
```

Riprova la richiesta sulla pizza. La formulazione è generata e varierà. Il rephraser interessa ogni risposta che porta `metadata: rephrase: True`, incluse molte risposte dei pattern di default. Tieni i messaggi esatti e regolamentati fuori da questo percorso generativo.

5. Aggiungi la validazione del trasferimento

Usa tre livelli di validazione:

- validazione nel domain per una regola di formato IBAN riutilizzabile;
- validazione nel flow per il limite di €5000 di questo processo di trasferimento;
- una custom action per il saldo del conto detenuto dal backend.

5.1 Valida l'IBAN nel domain

Modifica `lumera-assistant/domain/transfer_money.yml`.

Sostituisci lo slot `recipient_iban` con:

```

recipient_iban:
  type: text
  mappings:
    - type: from_llm
    # Universal format rule: an Italian IBAN starts with IT, then two check
    # digits, 27 characters in total. True wherever this slot is used, so
    # the rule lives at the domain level.
  validation:
    rejections:
      - if: not (slots.recipient_iban matches "^IT[0-9]{2}[A-Za-z0-9]{23}$")
        utter: utter_invalid_iban

```

Sotto `responses:`, aggiungi:

```

utter_invalid_iban:
  - text: "That doesn't look like an Italian IBAN – it should start with IT and be 27
    characters long. Could you check it?"

```

Il predicato rifiuta il valore candidato. Lo step di collect chiede poi nuovamente l'IBAN.

5.2 Valida il limite del trasferimento nel flow

Modifica:

`lumera-assistant/data/flows/transfers/transfer_money.yml`

Sostituisci lo step di collect di `transfer_amount` con:

```

  - collect: transfer_amount
    description: the transfer amount in euros; extract only the numerical value,
    ignoring any currency name or symbol
    # This process's ceiling, not the world's: the per-transfer limit
    # belongs to this flow, so it lives on this collect step.
    rejections:
      - if: slots.transfer_amount > 5000
        utter: utter_transfer_over_limit

```

In `lumera-assistant/domain/transfer_money.yml`, aggiungi questa risposta sotto `responses:`:

```

utter_transfer_over_limit:
  - text: "I can't set that up: online transfers are limited to €5000 per operation.
    For larger amounts, please visit a branch."

```

Questa regola appartiene al flow perché è un limite di questo processo, non una proprietà di ogni slot numerico.

5.3 Valida i fondi disponibili con un'action

Crea:

```
lumera-assistant/actions/validate_transfer_amount.py
```

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ValidateTransferAmount(Action):
    """Layer-3 validation: the truth about available funds lives in the
    core-banking system, not in YAML. Rasa runs any action named
    validate_<slot_name> automatically whenever that slot is collected."""

    def name(self) -> Text:
        return "validate_transfer_amount"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        amount = tracker.get_slot("transfer_amount")
        if amount is None:
            return []

        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        try:
            response = requests.get(
                f"{api_base}/v1/balance",
                params={"account_type": "checking"},
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            balance = response.json()["balance"]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            # If the balance service is unreachable, don't block the customer
            # at validation: the transfer execution itself fails clean.
            return []

        if amount > balance:
            dispatcher.utter_message(
                response="utter_insufficient_funds",
                available_balance=f"{balance:.2f}",
            )
            return [SlotSet("transfer_amount", None)]

        return []

```


Rasa esegue automaticamente un'action chiamata `validate_<slot_name>` dopo aver raccolto quello slot. Azzerare lo slot con `SlotSet("transfer_amount", None)` fa sì che lo step di collect lo richieda di nuovo.

L'action fallisce in modo permissivo (fail open) se la consultazione del saldo non è disponibile. La successiva action di esecuzione resta responsabile di impedire e segnalare un trasferimento fallito.

In `lumera-assistant/domain/transfer_money.yml`, aggiungi questa risposta sotto `responses:`:

```
utter_insufficient_funds:
  - text: "That's more than what's available on your checking account (€
{available_balance}), so this transfer would not go through. What amount should I use
instead?"
```

Sostituisci il blocco finale `actions:` con:

```
actions:
  - action_execute_transfer
  - validate_transfer_amount
```

Valida il progetto, poi riallena e riavvia:

```
cd lumera-assistant
.venv/bin/rasa data validate
cd ..
make run-rasa
```

Testa ciascun rifiuto:

```
IBAN:  BANANA123      → rejected; the assistant asks again
IBAN:  IT60X0542811101000000123456 → accepted
Amount: 9000          → rejected by the flow limit
Amount: 2000          → rejected because the demo balance is €1250
Amount: 300           → accepted
```

Le rejection YAML vengono eseguite prima della custom action di validazione, così i controlli locali poco costosi avvengono prima della chiamata al backend.

6. Costruisci la suite di regressione end-to-end

Costruisci la suite un file di test alla volta. Crea ogni directory quando aggiungi il suo primo file.

6.1 Aggiungi una fixture di autenticazione condivisa

Crea `lumera-assistant/e2e_tests/conftest.yml`:

```

# Fixtures visible to every test file under e2e_tests/.
# Slots listed here are pre-set right after session start, so test cases
# for sensitive flows can bypass the security-code challenge.
fixtures:
  - authenticated_user:
    - authenticated: true

```

La fixture aggira l'autenticazione nei test che si concentrano su un altro comportamento.

6.2 Testa i repair pattern

Crea `lumera-assistant/e2e_tests/patterns/cancellation.yml`:

```

test_cases:
  - test_case: cancellation_mid_transfer_pops_the_flow
    fixtures:
      - authenticated_user
    steps:
      - user: I'd like to make a transfer
        assertions:
          - flow_started:
              operator: all
              flow_ids:
                - transfer_money
      - user: Marco Bianchi
        assertions:
          - slot_was_set:
              - name: recipient_name
                value: Marco Bianchi
      - user: forget it, cancel the whole thing
        assertions:
          - flow_cancelled:
              flow_id: transfer_money

```

Crea `lumera-assistant/e2e_tests/patterns/chitchat.yml`:

```

# The rephrased chitchat answer is generated fresh on every run, so this
# case asserts the flow and response action – never the exact response text.
# The action assertion also proves that our override replaced the default,
# which executes utter_cannot_handle instead.
test_cases:
  - test_case: off_topic_message_goes_to_pattern_chitchat
    steps:
      - user: do you like pizza?
        assertions:
          - flow_started:
              operator: all
              flow_ids:
                - pattern_chitchat
          - action_executed: utter_free_chitchat_response

```

Il testo della risposta è generato, quindi il test non lo asserisce. Verifica invece sia la decisione di routing sia l'action di risposta. L'action distingue questo override dal `pattern_chitchat` di default, che esegue `utter_cannot_handle`.

Crea `lumera-assistant/e2e_tests/patterns/clarification.yml`:

```
# Which candidates the LLM lists is a judgement call that varies run to
# run (sometimes card_fees_info joins the two card flows) – and the `all`
# operator requires the exact option set. `any` pins what must hold: the
# pattern fired and offered card flows.
test_cases:
  - test_case: ambiguous_card_request_triggers_clarification
    steps:
      - user: something's wrong with my card
        assertions:
          - pattern_clarification_contains:
              operator: any
              flow_ids:
                - block_card
                - replace_card
```

`operator: any` consente all'LLM di includere altri candidati plausibili, pur continuando a verificare che i card flow siano stati offerti.

Crea `lumera-assistant/e2e_tests/patterns/correction.yml`:

```

# Correction at the recap: the amount must end at the corrected value.
# validate_transfer_amount is stubbed empty ("validation passes") – stub
# runs must stub every custom action the case touches.
stub_custom_actions:
  validate_transfer_amount:
    events: []
    responses: []

test_cases:
- test_case: correction_at_the_recap_updates_the_amount
  fixtures:
    - authenticated_user
  steps:
    - user: I want to send money to Marco Bianchi
      assertions:
        - flow_started:
            operator: all
            flow_ids:
              - transfer_money
    - user: IT60X0542811101000000123456
      assertions:
        - bot_uttered:
            utter_name: utter_ask_transfer_amount
    - user: "200"
      assertions:
        - bot_uttered:
            utter_name: utter_ask_transfer_confirmed
    - user: actually, make it 150
      assertions:
        - flow_started:
            operator: all
            flow_ids:
              - pattern_correction
        - slot_was_set:
            - name: transfer_amount
              value: 150.0

```

Crea `lumera-assistant/e2e_tests/patterns/nlu_trigger_coexistence.yml`:

```

# Coexistence guard: a high-confidence balance message must keep starting
# check_balance (via the nlu_trigger or the LLM – the assertion holds for
# either routing path). The balance fetch is stubbed.
stub_custom_actions:
  action_fetch_balance:
    events:
      - event: slot
        name: current_balance
        value: 1250.0
      - event: slot
        name: balance_fetch_ok
        value: true
    responses: []

test_cases:
  - test_case: high_confidence_balance_message_starts_check_balance
    steps:
      - user: what's my balance
        assertions:
          - flow_started:
              operator: all
              flow_ids:
                - check_balance
      - user: checking
        assertions:
          - bot_uttered:
              utter_name: utter_current_balance

```

6.3 Testa il percorso del trasferimento

Crea `lumera-assistant/e2e_tests/transfers/transfer_happy_path.yml` :

```

# The transfer happy path, with the core-banking call stubbed: this file
# tests the conversation logic, not the backend – it must pass with the
# mock API down.
#
# This case uses the step-script format (user turns + expected responses
# and slots) rather than per-turn assertions: it freezes the whole
# journey, and only checks our own literal responses – never rephrased
# pattern text.
stub_custom_actions:
  action_execute_transfer:
    events:
      - event: slot
        name: transfer_id
        value: TRF-0042
      - event: slot
        name: transfer_ok
        value: true
    responses: []
# Stub runs must stub EVERY custom action the case touches – including
# validate_transfer_amount, which the collect machinery calls
# automatically. An empty stub means "validation passes".
validate_transfer_amount:
  events: []
  responses: []

test_cases:
- test_case: transfer_happy_path_completes
  fixtures:
    - authenticated_user
  steps:
    - user: I want to send money to Marco Bianchi
    - slot_was_set:
      - recipient_name: Marco Bianchi
    - utter: utter_ask_recipient_iban
    - user: the IBAN is IT60X0542811101000000123456
    - utter: utter_ask_transfer_amount
    - user: "100"
    - slot_was_set:
      - transfer_amount: 100.0
    - utter: utter_ask_transfer_confirmed
    - user: yes, go ahead
    - utter: utter_transfer_done
    - user: no thanks
    - utter: utter_transfer_wrap_up

```

Gli stub mantengono il test focalizzato sul comportamento conversazionale. Un file di test con stub deve stubbare ogni custom action che i suoi casi chiamano, incluse le action di validazione invocate automaticamente.

6.4 Testa la validazione dell'input

Crea `lumera-assistant/e2e_tests/validation/transfer_input_validation.yml`:

```
# The two YAML validation layers, each rejected once and then accepted.
# The layer-3 validate_transfer_amount action is stubbed empty here
# ("validation passes") – its insufficient-funds catch needs the
# core-banking API up, so it is exercised live against the mock instead.
```

stub_custom_actions:

validate_transfer_amount:

events: []

responses: []

test_cases:

- **test_case:** garbage_iban_is_rejected_then_valid_iban_accepted

fixtures:

- authenticated_user

steps:

- **user:** I want to send money to Marco Bianchi

assertions:

- **flow_started:**

operator: all

flow_ids:

- transfer_money

- **user:** the IBAN is BANANA123

assertions:

- **bot_uttered:**

utter_name: utter_invalid_iban

- **bot_uttered:**

utter_name: utter_ask_recipient_iban

- **slot_was_not_set:**

name: recipient_iban

value: BANANA123

- **user:** sorry, IT60X0542811101000000123456

assertions:

- **slot_was_set:**

name: recipient_iban

value: IT60X0542811101000000123456

- **test_case:** amount_over_the_transfer_limit_is_rejected

fixtures:

- authenticated_user

steps:

- **user:** I want to send money to Marco Bianchi

assertions:

- **flow_started:**

operator: all

flow_ids:

- transfer_money

- **user:** IT60X0542811101000000123456

assertions:

- **bot_uttered:**

utter_name: utter_ask_transfer_amount

- **user:** "9000"

assertions:

- **bot_uttered:**

utter_name: utter_transfer_over_limit

```
- user: ok, 900 then
  assertions:
    - slot_was_set:
      - name: transfer_amount
        value: 900.0
```

Il comportamento in caso di fondi insufficienti non è incluso qui: stubbare il suo rifiuto testerebbe soltanto lo stub. Testa quell'integrazione manualmente, con la mock bank in esecuzione, come nella sezione 5.

6.5 Aggiungi i comandi di test

Aggiungi in coda al `starting-point/Makefile` questi target:

```
# Run the regression suite against the latest trained model.
# Stubbed custom actions are a beta feature, hence the env flag. Every turn
# still goes through the real LLM – a full run costs a few dozen calls.
run-e2e-tests:
  cd lumera-assistant && RASA_PRO_BETA_STUB_CUSTOM_ACTION=true .venv/bin/rasa test
e2e e2e_tests/

# Same suite, plus the flow/step coverage report.
run-e2e-coverage:
  cd lumera-assistant && RASA_PRO_BETA_STUB_CUSTOM_ACTION=true .venv/bin/rasa test
e2e e2e_tests/ --coverage-report
```

Il flag beta abilita gli stub delle custom action. I turni dell'utente passano comunque attraverso il command generator reale.

Esegui la suite da `starting-point/`:

```
make run-e2e-tests
```

Il routing dell'LLM è non deterministico. Un fallimento può rappresentare una reale variazione di routing anziché un errore di sintassi; ispeziona l'assertion fallita prima di modificare il test.

Genera poi la coverage:

```
make run-e2e-coverage
```

La coverage identifica flow e branch non testati. Il flow di autenticazione resta non coperto perché la fixture condivisa lo aggira; anche il portfolio più ampio e i branch di fallimento del trasferimento necessitano di casi successivi. Tratta questo report come un elenco concreto di test mancanti.

7. Rivedi il risultato

Hai aggiunto:

- la protezione dalle interruzioni per lo step di collect del codice di sicurezza;
- un pattern di chitchat personalizzato e la riformulazione contestuale;
- la validazione del trasferimento a livello di domain, di flow e di backend;
- test di regressione per i repair pattern, la coesistenza, la validazione e il percorso del trasferimento;
- i comandi per eseguire la suite e il suo report di coverage.

Confronta la tua `starting-point/` con `end-result/`. A parte gli artefatti di runtime locali e innocue differenze di ordinamento all'interno delle mappe YAML, entrambi i progetti dovrebbero ora contenere la stessa configurazione, le stesse action e gli stessi test.